



**Software Engineering Department
Braude College
Capstone Project Phase B**

Lemon Health Monitoring

By:

David Kotler
Maxim Cherepanov

Advisor:

Dr. Naomi Unkelos Shpigel

Project Code:

25-2-D-12

Link to GitHub:

<https://github.com/semaximche/Capstone-Project-Lemon-Health-Monitor>

Table of Contents

Abstract	3
1. Introduction	3
2. Model Training and Testing	4
2.1 Datasets and General Training	4
2.2 YOLOv11 Model Training	6
2.2.1 Leaf Detection	6
2.2.2 Fruits Detection	8
2.3 EfficientNet Model Training	9
2.4 Model Evaluation and Practical Results	11
2.5 Model Training Guide	14
3. Application Backend	15
3.1 Inference	15
4. Application Frontend	16
5. Application User Guide	19
5.1 Backend User Guide	19
5.2 Frontend User Guide	20
6. Results	21
6.1 Evaluation	21
6.2 Achievements	21
Academic and Technological References	23

Abstract

In our project and the previous Phase A paper we planned to develop a lemon tree health smart camera. An application that is able to take camera photos and use AI models predict what symptoms the tree is displaying. The application we have built uses a web interface based on **React** and **Tailwindcss** with a **FastAPI** backend server to run the models on and save results. The photo input for our application is passed to a **YOLOv11** object detection model to identify and isolate leaves from the image and each separate leaf cropped image is sent to an **EfficientNet** classifier model to get the probable symptoms and diseases in weights. We have received positive evaluation from peers who have tried our application in real world use regarding its accuracy and ease of use. Overall we have achieved most of the results we had set in Phase A during the development of our final project.

1. Introduction

Lemon trees face several common issues, beginning with dehydration and nutrient deficiencies, which can often be recognized visually through leaf symptoms and can significantly reduce yields. Beyond these basic problems, citrus plants are vulnerable to a range of bacterial, fungal, and viral diseases. Some easily identifiable, such as sooty mold or bacterial blight, and others that mimic dehydration symptoms or require internal inspection, such as Mal Secco. For this project, the focus is on key diseases including **anthracnose, citrus canker, curl virus, nutrient deficiencies, dehydration, sooty mold, spider mites, and bacterial blight**, alongside the characteristics of a healthy leaf.

From our literary overview in **Phase A** we have presented existing literature that shows that automated plant disease identification using machine learning has achieved high accuracy across various crops, with multiple studies reaching **94 to 99% accuracy** using **CNNs**[\[1\]](#). Research specific to lemon leaves has demonstrated similarly strong performance, though real-world challenges remain due to small datasets and the use of idealized images[\[2\]](#). Some citrus-disease studies rely on isolated leaves photographed against blank backgrounds, which highlights the gap between controlled research settings and actual orchard conditions[\[3\]](#).

To address these limitations, the project aims to combine object detection and classification using a two-stage approach: first isolating lemon leaves in an image with a **YOLOv11**[\[4\]](#) **model**, then diagnosing diseases using an optimized **EfficientNet model**[\[5\]](#) trained on leaf images on blank backgrounds. The system's output can be enhanced with an LLM to deliver treatment recommendations in natural language. All models will be evaluated using standard metrics including accuracy, precision, recall, and F1 score to ensure both technical performance and practical field usability.

2. Model Training and Testing

2.1 Datasets and General Training

As described in Phase A we want to combine two datasets, one for identifying leaves within photos of lemon trees, the second for identifying actual symptoms and diseases recognizable within the leaves. For the first task of identifying leaves we have managed to find a Kaggle dataset created and annotated by Alex Olariu for YOLOv11 models[\[6\]](#).



The leaves dataset includes a variety of different plant leaves in various environments and conditions. Some outside in agricultural settings, some on blank backgrounds, some include fruits too. While not specifically suitable for lemon trees the high variability might help identify leaves in non-ideal conditions that can often come from taking photographs outside.

For the second task we used a dataset published on ScienceDirect called “**A comprehensive image dataset for the identification of lemon leaf diseases and computer vision applications**”[\[3\]](#) that includes classification to 9 different categories of symptoms and diagnosis.



The diseases dataset contrary to the leaves dataset is much more uniform. All the lemon leaves are sat on a blank background and the disease symptoms are intentionally shown to help the model more accurately detect them. Since we only run the model on isolated leaves detected by the leaves model we can use a more focused and uniform dataset to help the training fine tune the model as much as possible at the slightest differences in the images.

An additional dataset for citrus fruits was used, taken from **Roboflow**[\[7\]](#) as a late addition to the algorithm to remove fruits detected as leaves by our model.

For the training itself we have used the built-in YOLOv11 train method and a custom made EfficientNet models trainer both using python scripts. The models were trained several times using different image sizes, epochs number, learning rate and momentum rate and the final models used in our application were the best overall when taken against practical tests with the full algorithm pipeline.

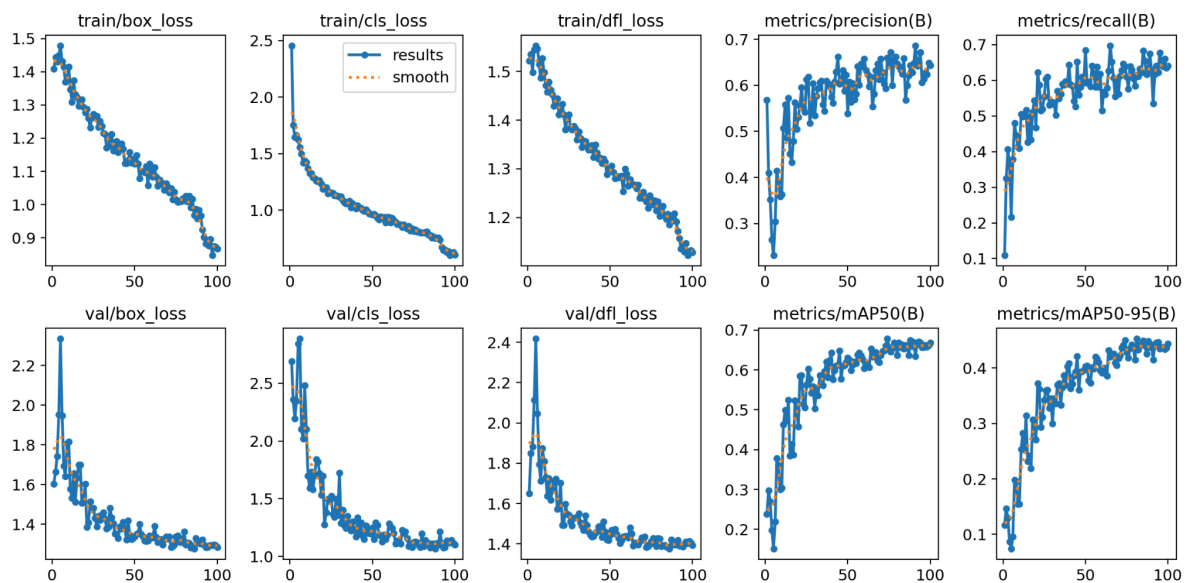
2.2 YOLOv11 Model Training

2.2.1 Leaf Detection

After training an initial YOLOv11 for 100 epochs on a dataset for isolating leaves from wide images. Running over a sample of realistic input photos, the model can correctly isolate and show us where each leaf is located but has a tendency to also select lemon fruits as leaves. One possible solution(which we eventually implemented) is to train a secondary model to also identify and isolate fruits and discard overlapping predictions of both leaves and fruits, therefore getting only leaves.

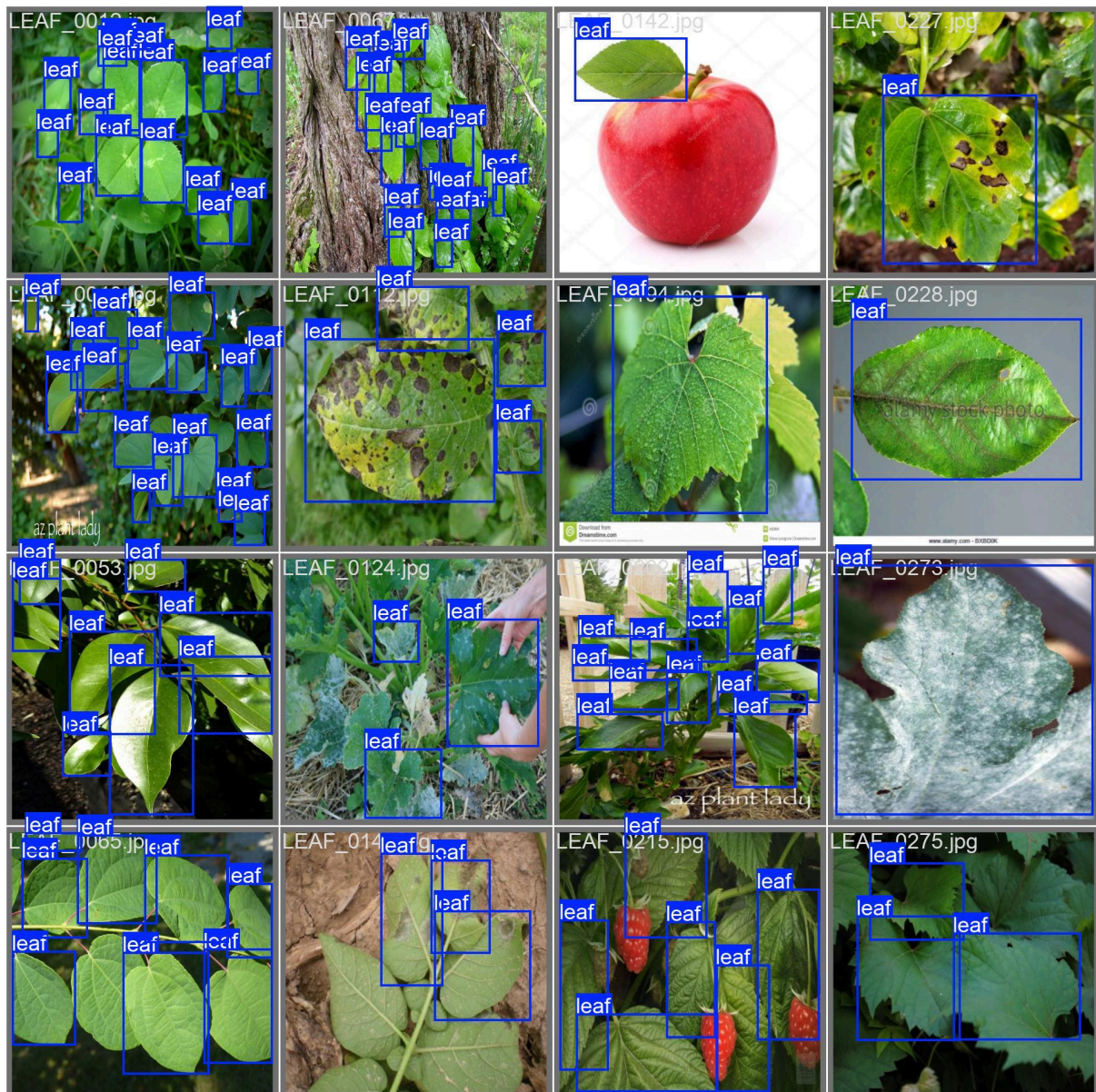
It is important to note that YOLOv11 uses partial batching, meaning each epoch only trains on a small subset of the full dataset. Such that we would need more epochs of training to back propagate the weights on results from the complete dataset. 50 epochs roughly equal to training over the whole dataset 4 times.

We've tested several different runs with different parameters and found that training on 100 epochs with image size of 640x640 yielded the best results with the least high confidence false positives for the the leaves detection:



Since this is an object detection model that recognizes boxes of objects like leaves in our case, the mAP (mean average precision) statistic displayed above gives us the best general performance score testing for what percentage of labels the model accurately predicted within a 50% overlap zone. Our model specifically reached mAP50 of 65-70% detecting leaves.

Test images results for the YOLO model:

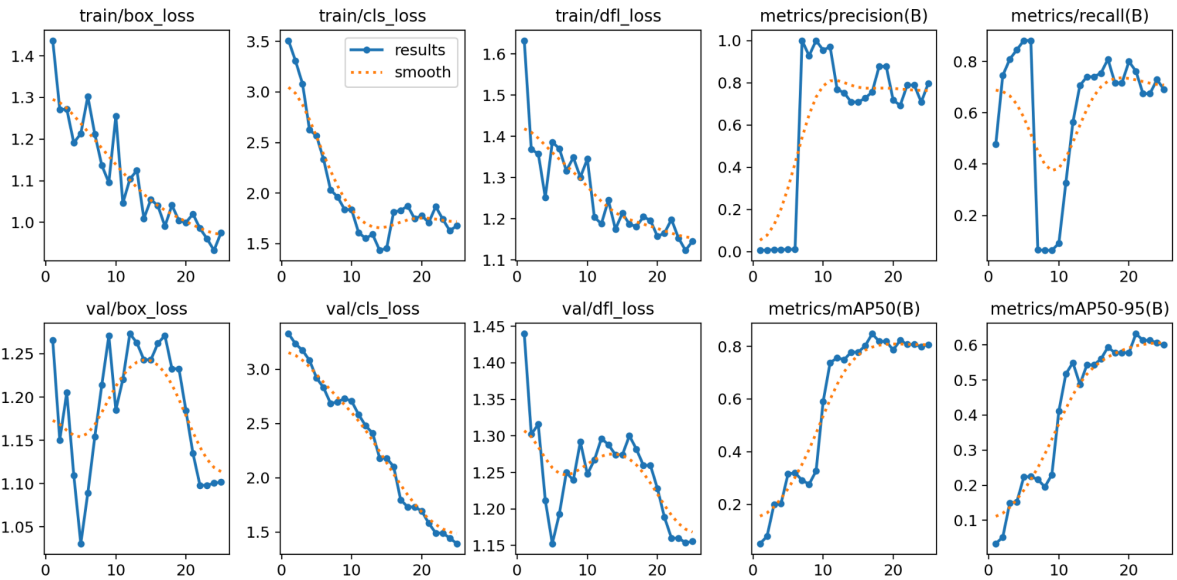


Throughout the training and testing phase we had found out the YOLOv11 leaves detection model often marks lemon fruits as leaves which often get classified as dry leaves (most likely because they are yellow coloured like dry leaves)

To fix that we added a second YOLOv11 model trained on looking specifically for lemon trees. After detecting both leaves and lemon fruits, our algorithm removes any object identified as leaf that also overlaps with an object identified as a fruit.

2.2.2 Fruits Detection

The fruit model was trained on 100 epochs with an image size of 1024x1024. Due to time constraints no fine tuning on the parameters was done but the initially trained model performed well enough to help discard most of the fruit detections. The final results of the training were:



Same as before the most important metric to look at is the mAP50 which sits at 80% meaning 80% of the time the model accurately detected lemon fruits in the image given a 50% overlap threshold in the box.



While the fruits model isn't fine tuned and isn't as good as it can get, it still detects fruits accurately enough for our purposes. One example seen in the example batch above is that the model does not detect cut fruits (seen in the lower middle image) but this does not affect us since we're only looking to detect fruits on trees which the model can do fairly accurately.

It is important to note the fruits model differentiates between ripe and unripe fruits due to how the dataset used was structured. We had found that leaves often get labeled as unripe fruits and as such we only use the ripe fruits detection for our final application.

2.3 EfficientNet Model Training

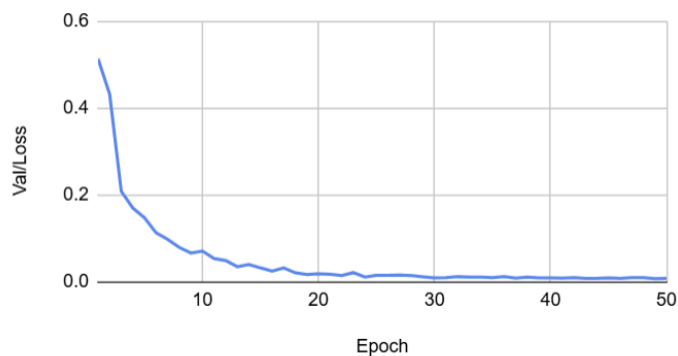
After an initial EfficientNet model training for 50 epochs on the dataset classifying lemon leaves on blank backgrounds into the following categories: **Anthraco**nose, **Bacterial Blight**, **Citrus Canker**, **Curl Virus**, **Deficiency Leaf**, **Dry Leaf**, **Healthy Leaf**, **Sooty Mold**, and **Spider Mites**. After running over a sample of results from the previously trained YOLOv11 model we have reached the following conclusion on what the AI can and cannot identify accurately.

1. **Anthraco**nose - Characterised by small black spots on leaves, on a sample test image, the EfficientNet model correctly identified and classified the leaves into the disease
2. **Citrus Canker** - Characterised by small brown spots in yellow circles, the model predicted correctly for some of the obvious leaves, but classified the others as healthy
3. **Dry Tree/Dehydration** - The model correctly identifies both completely dry trees and trees with slight yellowing indicating dehydration.
4. **Curl Virus** - Identified by curled up dry looking leaves. The model has classified these leaves into a variety of different symptoms and diseases but not curl virus itself.
5. **Sooty Mould** - Similar to curl virus, a variety of diseases were classified but not sooty mold itself.
6. **Iron/Magnesium/Zinc/Nitrogen Deficiencies** - Similar to curl virus and sooty mould. No single categorisation for these leaves.

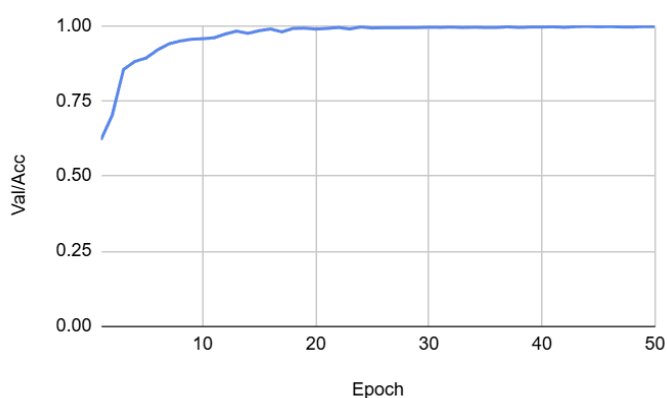
For the most part, the model correctly identifies healthy leaves from healthy trees, dry and dehydrated leaves and is good at identifying very obvious diseases that are characterised by black/brown spots and yellow circles, but not less obvious diseases and symptoms such as sooty mould and deficiencies. For less obvious trees with some sort of disease the model typically assigns a variety of different categories for each leaf but almost never assigns healthy.

The EfficientNet model was trained on 50 epochs on image size of 384x384 which is the given default size for optimal training. Training was done with an exponential lr scheduler with very slow initial learning rates (learning rate 0.0001 momentum 0.99) and we have reached the following results:

Val/Loss vs. Epoch



Val/Acc vs. Epoch



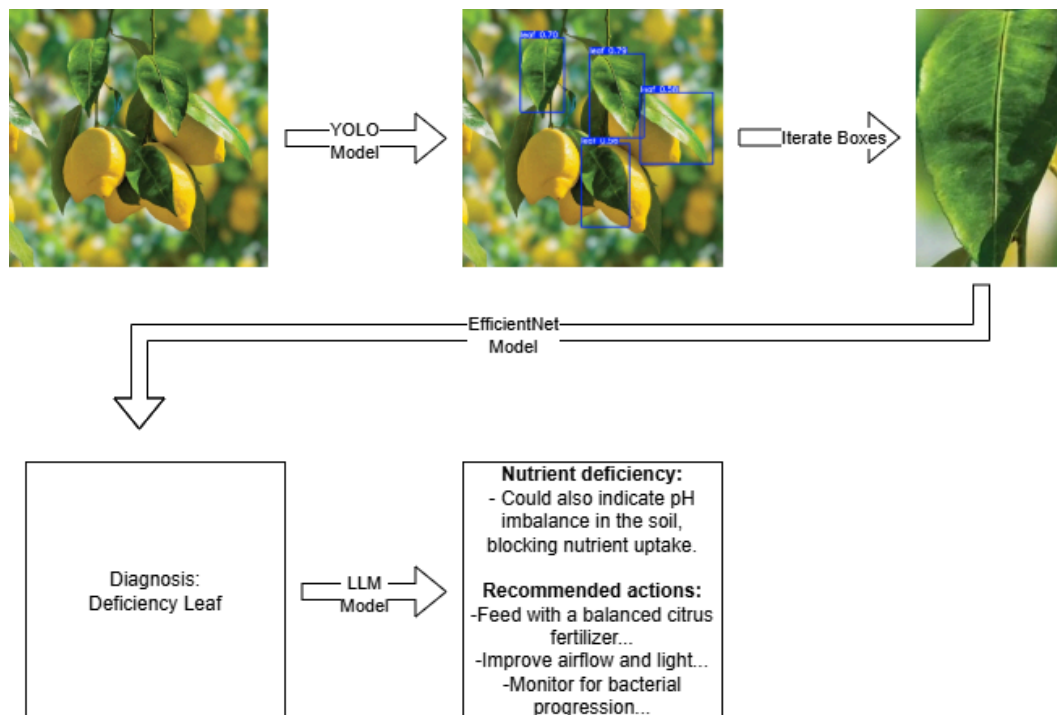
Accuracy reaching above 90% and 98% 99% towards the last epochs shows us that the model can very efficiently be trained on similar disease datasets for other plants very quickly. In practice despite the high accuracy the model still misses when given non test images. Mostly when it sees symptoms and diseases it was not trained on at all often giving diverse classifications on lemon trees that it cannot recognize the disease. (For example labeling one leaf as Citrus Canker, another leaf as Sooty Mould and another leaf as Spider Mites despite the plant visually displaying a disease not trained on)

Despite already using low learning rates it is possible pushing the learning rate even further low is needed in order to get more optimal results. Most likely, due to how similar each image in the disease dataset is (single leaf on blank background), the training needs to be very sensitive to small changes on the input image suggesting the need for very low learning rates.

2.4 Model Evaluation and Practical Results

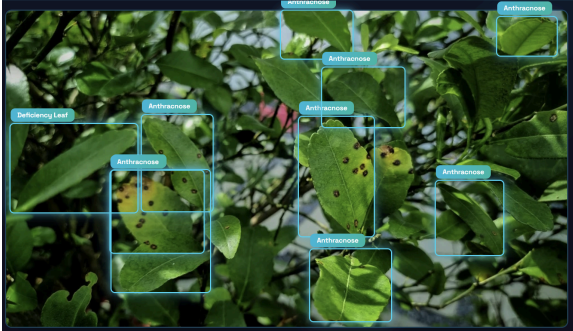
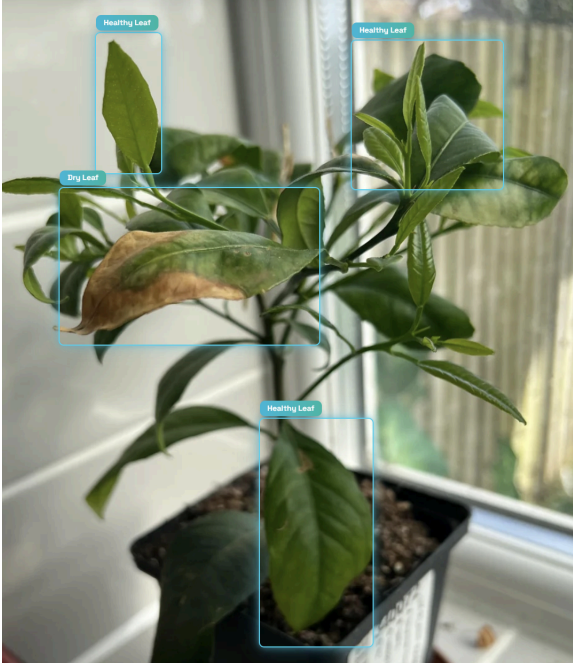
After the initial model training sequential training with different parameters did not give us better results and despite trying to fine tune. It is possible that more rigorous training on varying parameters is needed such as combining different schedulers, learning rate, momentum and image sizes or potentially different image transformations. On the other hand it is also possible that the only way to get better results is to create more comprehensive datasets. For one, the disease dataset we used is limited to only 9 labels and could be expanded to include more diseases and symptoms. The YOLO models trained on leaves and fruits could be trained on more focused datasets specific for photos of lemon trees at a distance instead of close up.

With the models we had implemented the analysis algorithm as described in our Phase A. The algorithm takes a photograph of a lemon plant as an input, isolates the leaves using the YOLO model and labels each leaf with a disease or symptom using the EfficientNet model:

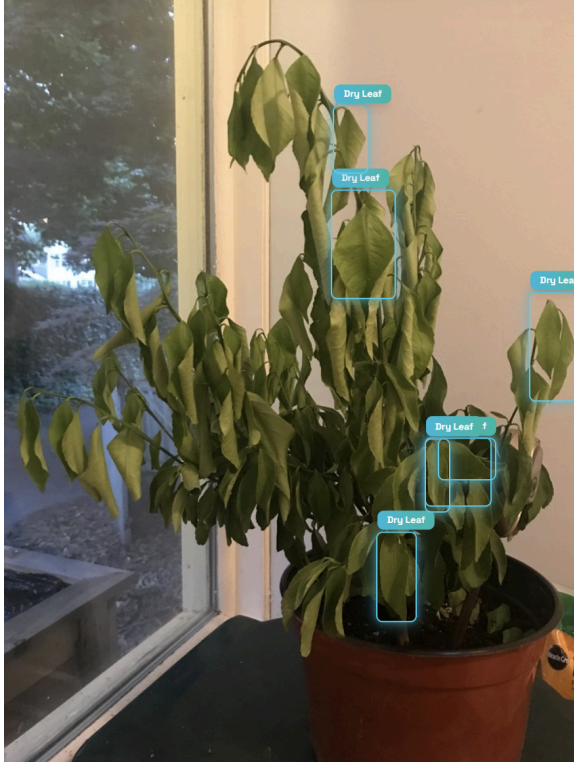


On practical usage the model shows various consistent patterns that can help understand the model outputs better.

Here are several patterns we've noticed in the model analysis while testing practical photos including where the model works best and what flaws we have found in the model:

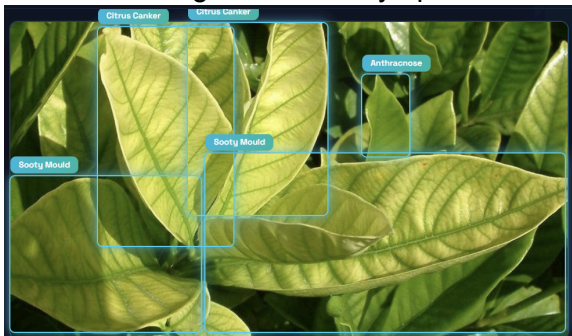
Obvious recognizable symptom in plant 	If the plant has a visually obvious disease that the model was trained on it will identify and label most leaves with that disease. For example in this photograph the plant is showing visible signs of Anthracnose and the model very accurately detects the leaves as such with the exception of one leaf detected as being "Deficient Leaf"
Obvious recognizable symptom in leaf 	Same as above if the symptom the model was trained on is present in a leaf that is very visible and recognizable the analysis output will be very accurate. In this instance the model labels the single dry leaf as such.

Inclination towards labeling dry and dehydrated leaves



In general the model is highly inclined to label dry leaves as such even if there's an underlying issue. In this case the soil is not suitable and lacks nutrients for the tree to grow properly. The LLM output is able to recognize that dry leaves are not only from dehydration but can also stem from soil issues and recommend checking soil pH and ensuring proper fertilization.

Diverse labeling of different symptoms



In this photo the plant is specifically deficient in iron but the model isn't trained to recognize this symptom. As such the model labels the leaves in a variety of different diseases showing that there is something wrong with the plant but not quite being able to pinpoint the issue. Our LLM is instructed to recognize this diverse labeling of diseases as a sign that the model cannot recognize the symptom and alert the user the model cannot confidently tell the problem with the plant.

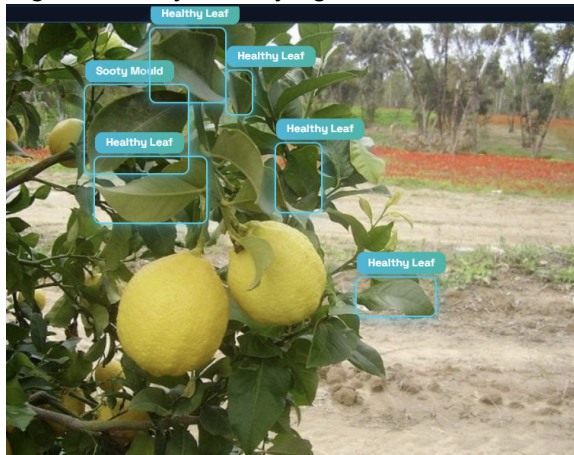
Handling large photos and fruits recognition



The algorithm is able to handle very large photos with many leaves isolating the most clear ones and giving them accurate diagnosis. One issue is that sometimes the fruits recognition fails in which case the disease recognition model almost always labels them as dry leaves (potentially because of the yellow colour)

In this photo there are only 3 leaves not accurately identified as healthy.

High accuracy identifying health leaves



While the model tries its best to find flaws in the photos it still very accurately identifies leaves that don't resemble any disease. In leaves that display any type of problem the model will almost always label a disease or symptom with varying accuracy.

In this photo the single leaf identified with Sooty Mould has signs of an insect while all the leaves that look perfectly healthy are labeled as such.

2.5 Model Training Guide

The final models are already included in the git project as such it is not necessary to train new models, in any case the following section describes the training process. For both trainers run in python using uv to automatically install the required python version and dependencies.

Download all the datasets from the following links:

- **Leaf Detection** - <https://www.kaggle.com/code/luisolazo/leaf-detection-w-ultralytics-yolov8-and-tflite>
- **Fruit Detection** - <https://universe.roboflow.com/braude-nqqje/citrus-exssv-nwvsu/dataset/1>
- **Disease Detection** - <https://www.sciencedirect.com/science/article/pii/S235234092401206X>

For the EfficientNet disease detection model in the **model-training** folder by default the *pyproject.toml* file is configured to use CUDA with Nvidia GPUs, for CPU training remove lines 15-21 or change to different CUDA versions according to the pytorch Getting Started page.

To run training with default settings use:

```
uv run mergedtrainer.py [DATASET_INPUT DIRECTORY]
```

Otherwise see other parameters and options using:

```
uv run mergedtrainer.py -h
```

For the YOLO leaf detection and fruit detection models in the **model-training** folder open and edit the file **yolo_training.py** and change parameters to your likings and then run using:

```
uv run yolo_training.py
```

3. Application Backend

While training the models we also started working on our application backend. Initially built as described in Phase A using **FastAPI**[\[8\]](#) written in Python. We implemented a fast working prototype for the backend and tested it using **Swagger UI**[\[9\]](#).

We have implemented all the features we had planned including LLM support to give our users human readable results and recommendations instead of just numbers and percentages of how confident our model is in the analysis.

The LLM support we ended up using was **Gemini Flash**[\[10\]](#) which gave us quick responses to our requests fitting in with our goal of delivering the final analysis as quickly as possible.

3.1 Inference

While the skeleton of the application backend was being written we had to come up with a way to run our algorithm's inference using the backend. Since the algorithms included running through several models before reaching the final output we had decided to separate the actual server backend and the **inference program**. Our solution to pass messages between the two was using **RabbitMQ**[\[11\]](#) that would allow the backend to publish inference tasks to the separate program that would run these tasks on the algorithm separately and notify the backend when a task has been completed to be passed on to the application frontend and the end user.

Overall after finalizing the architecture of the backend we have added one component between the algorithm and the backend, that component being RabbitMQ to send inference tasks and receive inference results.

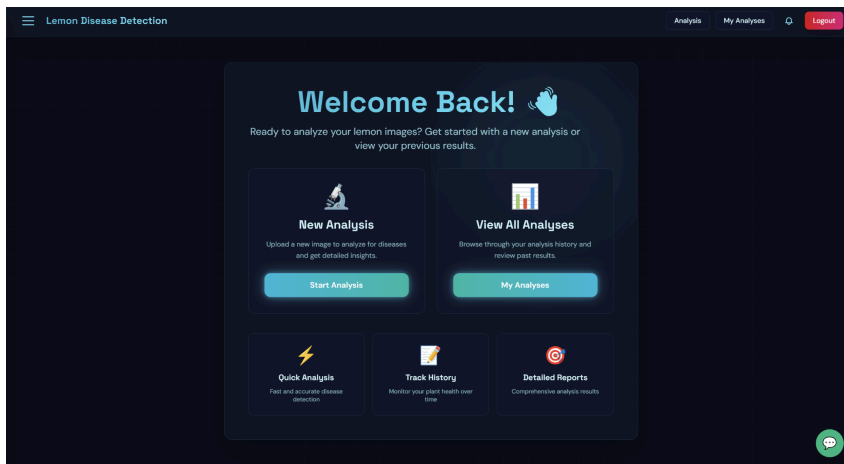
4. Application Frontend

Originally in our Phase A paper we had decided on using React to write the frontend but we didn't specify a framework to build our web application on. In the beginning of our work in this phase we have quickly settled on using **ReactRouter**[\[12\]](#) as our framework of choice since it was simplistic and quick to use and allowed multiple pages within our application.

The first features implemented were a basic home page, login page and the analysis page that allowed the users to upload and submit photos for analysis for our algorithm.

After several revisions we have implemented all the features we planned except admin panels and account management and tree numbers meaning that every analysis is isolated and our algorithm does not take into account past analysis of the same tree.

In the frontend we have implemented a basic landing page:



Signin and login functionality:

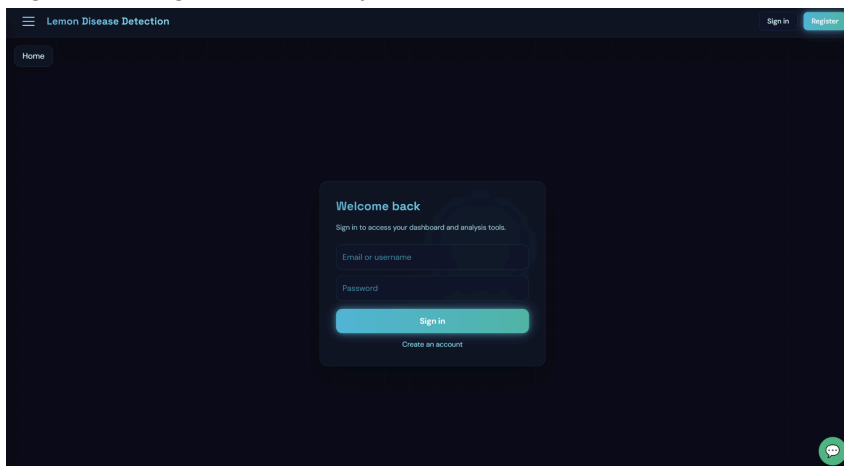
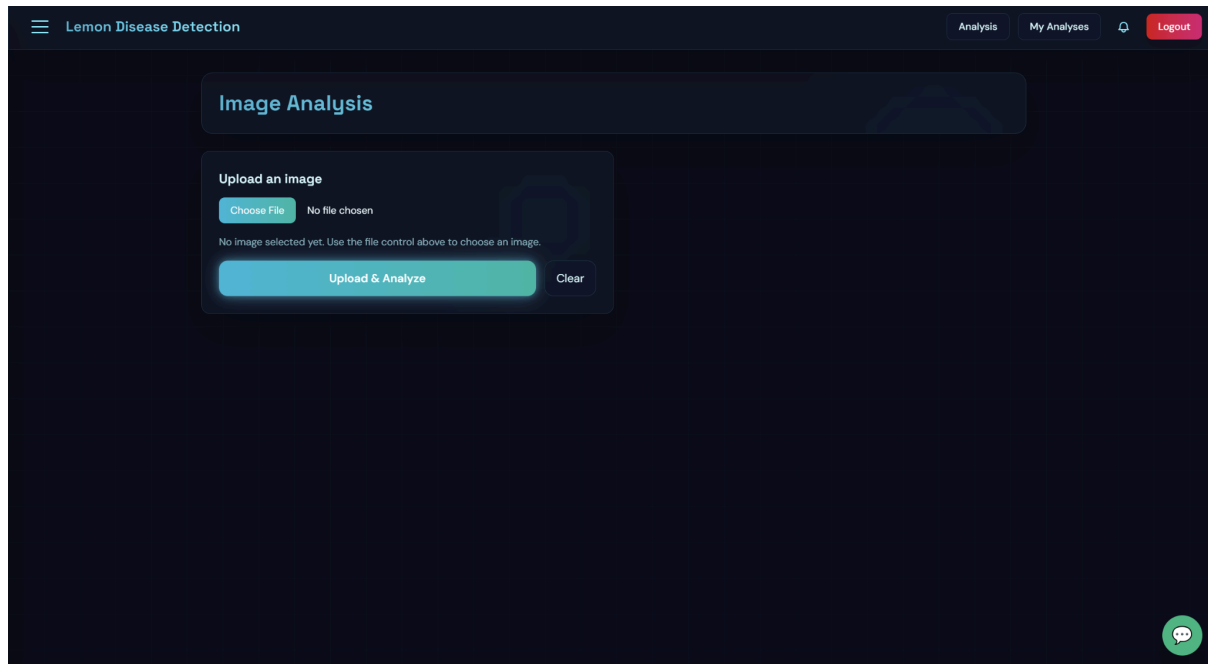
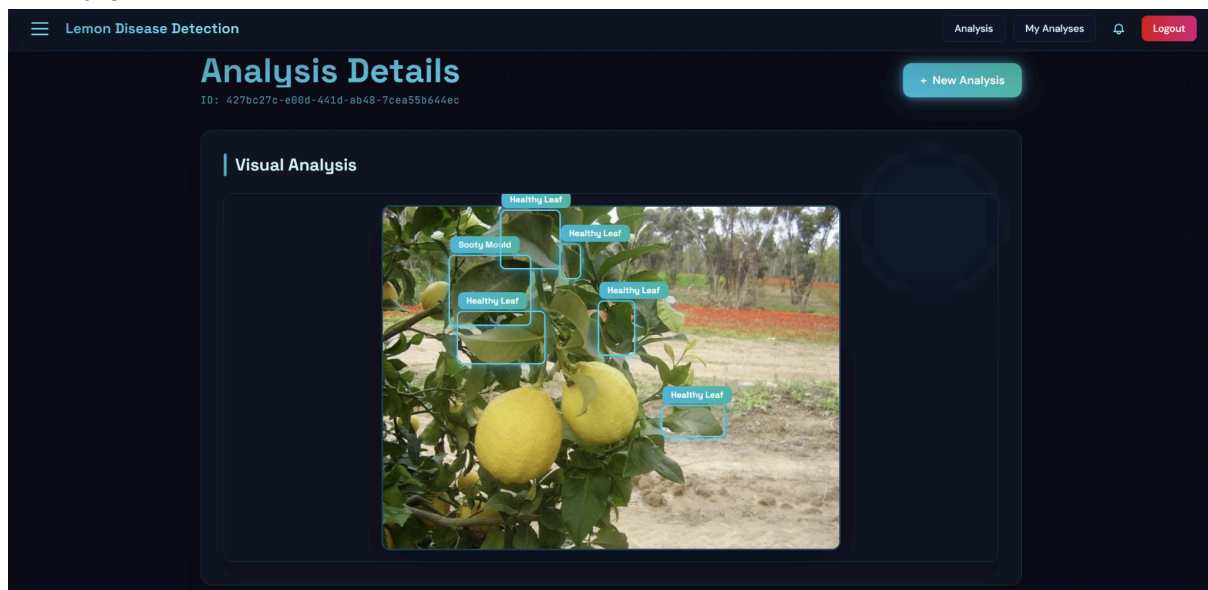


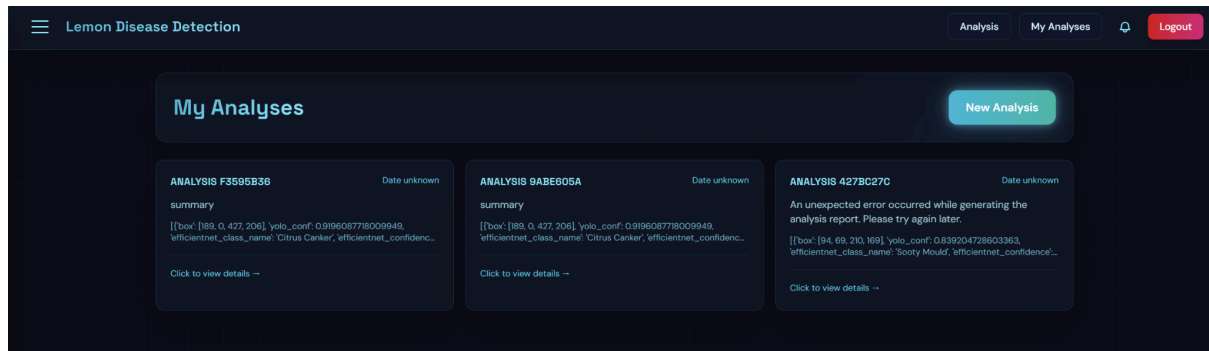
Photo analysis upload page:



Results page showing both the algorithm output as the diseases and symptoms are overlaid on the leaves they were detected on and an LLM output at the bottom explaining the results and giving recommendations for what to do next and how to ensure the plant's healthy growth:



Past analysis page for seeing everything the user have uploaded in the past:



While developing the frontend we focused on designing responsive layouts that can adjust on different screen sizes with special care towards phone displays and desktop displays.

In addition a focus was given on making the web pages light and fast to open on a variety of devices.

5. Application User Guide

The following section is dedicated to how to run the application on a desktop environment. The process is adaptable for a terminal environment when deploying. Model training as described in section 2.5 is not necessary as the models are already included in the git project.

5.1 Backend User Guide

The application runs on docker containers and as such installation of docker is required. On personal computers it is possible to use **Docker Desktop** to set up and run the application.

To setup the application from the git repository navigate to:

/Phase B/capstone

An **.env** file is required inside the above directory to store all keys and settings with the following contents filled in as necessary:

```
# Google Gemini API Key
GEMINI_API_KEY=

# JWT Settings
JWT_SECRET_KEY=
JWT_ALGORITHM=HS256
JWT_ACCESS_TOKEN_EXPIRE_MINUTES=30
JWT_REFRESH_TOKEN_EXPIRE_DAYS=7

# Storage Settings
STORAGE_BUCKET=analysis
STORAGE_ENDPOINT=users
STORAGE_ACCESS_KEY=
STORAGE_SECRET_KEY=

# RabbitMQ Settings
QUEUE_HOST=rabbitmq
QUEUE_USER=
QUEUE_PASSWORD=

# Google OAuth (Optional)
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
GOOGLE_REDIRECT_URI=

# Home Assistant (Optional)
HOME_ASSISTANT_URL=
HOME_ASSISTANT_TOKEN=

# App Settings
DEBUG=false
```

Open a terminal interface in the directory and run the command to build the docker containers:

```
docker compose up --build
```

If successful you will have a new composed docker stack called **capstone** with 3 containers inside:

- **lemon-inference** - The separate inference program which runs the models on inputs from the RabbitMQ queue.
- **lemon-api** - The actual backend server giving support to API requests.
- **lemon-rabbitmq** - RabbitMQ queue that manages inference tasks and notifications.

In addition to the containers, a volume will be created for all the data storage the application requires. By default the API will have a SwaggerUI available at <http://localhost:8000/docs#/> which can be used to test requests such as login, signup and analysis.

5.2 Frontend User Guide

The frontend application requires node.js to run in a desktop environment. It should be installed prior to running the application.

To run a basic frontend server from the git repository navigate to:

```
/Phase B/capstone-react
```

Either open an IDE capable of running node.js(such as visual studio code) or a terminal from the above folder.

Before running the frontend web server change the configuration as needed at:

```
/Phase B/capstone-react/app/lib/api-config.ts
```

Most importantly **API_BASE_URL** should point towards the actual backend server(for example <http://localhost:8000> if running locally)

To run the frontend web server run the command:

```
npm run dev           (for quicker dev mode)  
  
npm run build  
npm run start       (for running full build)
```

6. Results

6.1 Evaluation

Our completed program we had sent to several professionals between agricultural education professionals and other software engineers to evaluate based on their personal experiences and expertise with growing lemon trees.

On the SUS questionnaire we received good feedback on our applications' ease of use and simplicity. We also received good feedback on the responsiveness of the application and the fast analysis response. We did not receive explicit feedback on how to improve the ease of use of our application but seeing as the users who graded it lower were not from technological backgrounds we can infer that the way we display the analysis with percentages of confidence may have affected how less technologically knowledgeable users find the final results less clearly.

The people who evaluated our project have for the most part received accurate results from our models and didn't give any marks on improvements for the detection.

The overall SUS score for the application is **70.8** which is considered acceptable.

6.2 Achievements

During phase A of the project we had set several achievements that we were trying to achieve during the final project work. Out of our goals we have managed to achieve most of what we set to ourselves. In more details the exact goals and results that we had set are as follows:

1. Accurate plant analysis -

We had set our goal for model accuracy as >90% and we had achieved that goal for the individual models. In practical testing of the whole algorithm the results are lower and the system does misidentify some patterns as described in section 2.4.

2. Performative plant analysis -

We had set our performance goal to get our results within 5 seconds and we had achieved that goal. The final models and algorithm work quickly and efficiently even on sub-optimal hardware such as using CPU instead of GPU for the model inference. This result allows us to give our users quick and immediate feedback on their analysis request for our algorithm.

3. Modular and scalable system -

In terms of scalability of the models we had achieved our goal, we have a collection of efficient models that we can scale and train on other plants with similar data sets to our lemon plants ones. In terms of the final application it allows us to add more disease classes immediately as needed but not immediately compatible with adding more plant types and that would require additional work on the application.

4. User centric system -

We aimed to create a user friendly application that could be used even by people not proficient with technology and computers and from our questionnaire evaluation we had achieved that goal based on the positive feedback. Most of the users were satisfied with our application's ease of use. We aimed for an excellent score of 80 but our final result has only received an acceptable score of 70.8.

Academic and Technological References

1. Islam, M. M., Ahad, A., Talukder, M. A., Uddin, K., Uddin, M. A., Hasan, M. K., ... Debnath, S. K. (2023). DeepCrop: Deep learning-based crop disease prediction with web application. *Journal of Agriculture and Food Research*, 14, 100764–100764. <https://doi.org/10.1016/j.jafr.2023.100764>
2. Aktaruzzaman Pramanik, Khan, A. Z., Biswas, A. A., & Rahman, M. (2021). Lemon Leaf Disease Classification Using CNN-based Architectures with Transfer Learning. 2022 13th International Conference on Computing Communication and Networking Technologies (ICCCNT), 1, 1–6. <https://doi.org/10.1109/icccnt51525.2021.9579586>
3. Siam, K., Prayma Bishshash, Md Asraful Sharker Nirob, Mamun, S. B., Md Assaduzzaman, & Sheak. (2024). A Comprehensive Image Dataset for the Identification of Lemon Leaf Diseases and Computer Vision Applications. *Data in Brief*, 58, 111244–111244. <https://doi.org/10.1016/j.dib.2024.111244>
4. YOLOv11. (2025). Retrieved October 15, 2025, from YOLOv11.com website: <https://yolov11.com/>
5. Tan, M., & Le, Q. (2021). EfficientNetV2: Smaller Models and Faster Training. Retrieved from <https://arxiv.org/pdf/2104.00298>
6. Alex Olariu (2020). Leaf Detection dataset for detecting leaves on plants or in the trees. Retrieved from <https://www.kaggle.com/datasets/alexo98/leaf-detection>
7. Citrus dataset (2026). Retrieved from <https://universe.roboflow.com/braude-nggie/citrus-exssv-nwvsu/dataset/1>
8. FastAPI (2025). Retrieved October 15, 2025, from Tiangolo.com website: <https://fastapi.tiangolo.com/>
9. SwaggerUI (2026). Retrieved from <https://swagger.io/tools/swagger-ui/>
10. GeminiAI (2026). Retrieved from <https://gemini.google.com/app>
11. RabbitMQ (2026). Retrieved from <https://www.rabbitmq.com/>
12. ReactRouter (2026). Retrieved from <https://reactrouter.com/>